시스템 설계 문서(SDD)

프로젝트 제목 : AMR 협업형 실내 안전위협요소 선제 대응 시스템

날짜 : 2026.01.26

버전 : 0.7

---

1. 개요

본 시스템은 쇼핑몰 실내 환경에서 발생 가능한 안전위협요소(쓰레기, 액체류, 타일 파손 등)를 **선제적으로 탐지하고, 두 대의 자율 이동 로봇(AMR)이 협업하여 통제 및 사후 대응을 수행하는 PoC(Proof of Concept) 시스템**입니다.

기존 운영 환경에서는 안전위협요소가 발생한 시점부터 관리자가 인지하고 현장을 통제·조치하기까지 시간적 공백이 존재하며, 이 구간에서 2차 안전사고가 발생할 가능성이 있습니다. 본 시스템은 이러한 공백을 최소화하기 위해 **고정형 Webcam 기반 광역 감시 + AMR 기반 근접 확인 및 대응** 구조로 설계되었습니다.

Webcam 상에서 먼저 현장에 도착한 AMR(통제 로봇)이 위협요소를 분류·보고하고, 이후 다른 AMR(복구 로봇)이 상황 유형에 맞는 대응 장비를 경유하여 현장에 도착함으로써, **다중 AMR 협업 기반의 단계적 대응 시나리오**를 구현합니다.

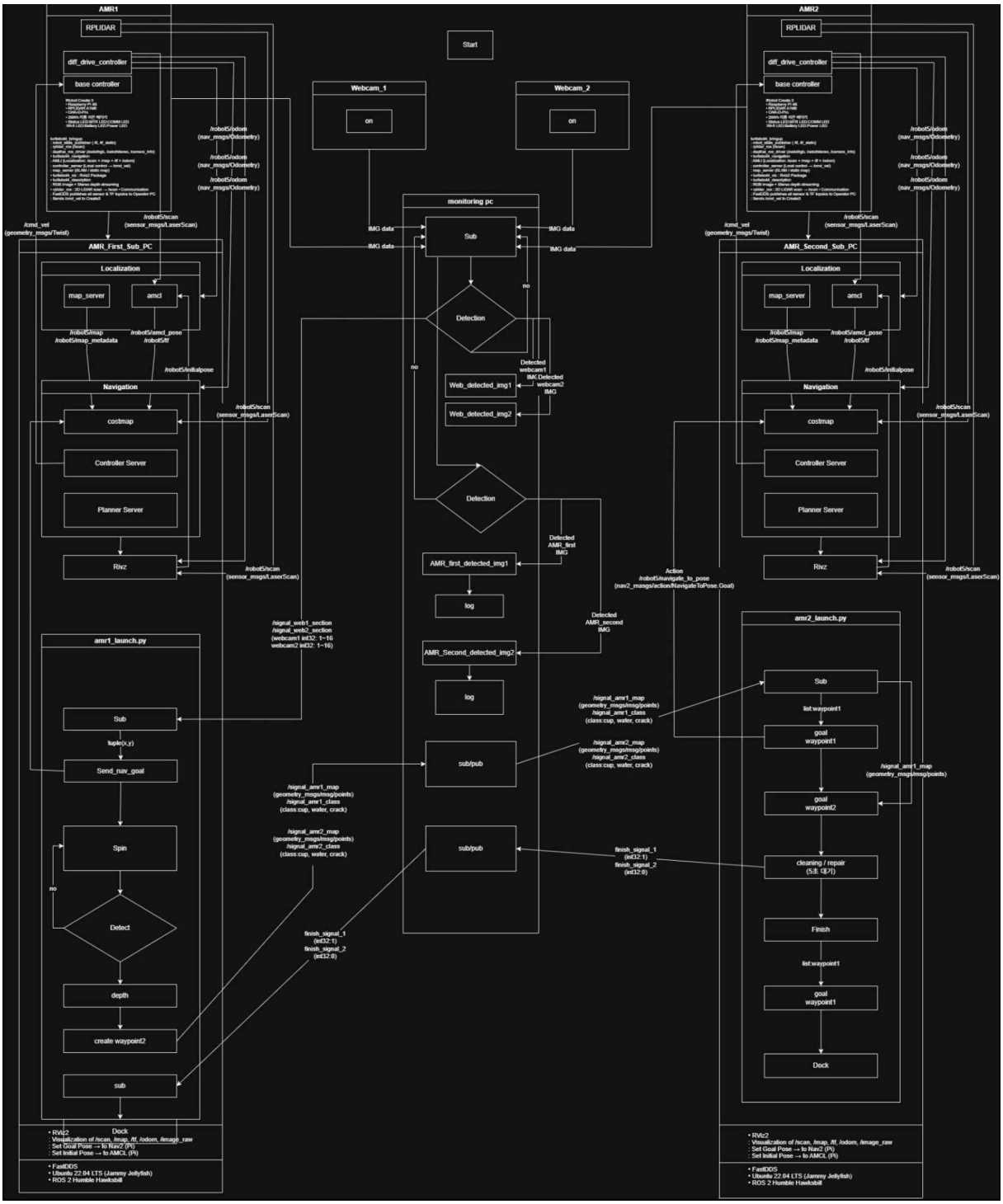
2. 시스템 아키텍쳐

본 시스템은 총 3대의 PC와 2대의 자율 이동 로봇(AMR)으로 구성되며, **중앙 관제 서버를 중심으로 한 Star Topology 기반 분산 구조**를 가진다. 모든 구성 요소는 동일한 내부 네트워크 환경에서 **ROS 2 DDS 기반 통신**을 통해 연동된다.

- **PC-3 (모니터링 서버)**: Webcam 영상 수신, 객체 탐지, 중앙 관제 및 임무 조율
- **PC-1 (AMR-1 제어 서버)**: 통제 로봇 제어 및 로컬 객체 탐지
- **PC-2 (AMR-2 제어 서버)**: 복구 로봇 제어 및 로컬 객체 탐지
- **AMR-1**: 1차 대응(통제 로봇)
- **AMR-2**: 2차 대응(복구 로봇)

고정형 Webcam에서 수집된 영상은 PC-3에서 실시간 객체 탐지를 수행하며, AMR에서 수집된 영상 데이터는 각 AMR 제어 PC(PC-1, PC-2)에서 로컬로 처리한다. 이를 통해 영상 스트리밍으로 인한 네트워크 지연을 최소화하고, 중앙 서버와는 이벤트 기반 메시지(Class, 위치, 상태)만을 교환하도록 설계하였다.

2.1 고레벨 아키텍처 다이어그램



3. 구성 요소 설계

3.1 하드웨어 구성 요소

1) 모니터링 서버(PC-3)
   a) CPU : 8-core 8 thread
   b) Ram : 32GB

c) GPU : NVIDIA RTX 3060
d) Camera : USB 카메라 * 2대
e) OS : Ubuntu 22.04
f) Network : Wi-Fi 6을 통한 고대역폭 무선 연결 지원

2) AMR 제어 서버(PC-1, PC-2)
a) CPU : 8-core 8 thread
b) Ram : 32GB
c) GPU : NVIDIA RTX 3060
d) OS : Ubuntu 22.04
e) Network : Wi-Fi 6을 통한 고대역폭 무선 연결 지원

3) 자율이동로봇(AMR-1, AMR-2)
a) 플랫폼: TurtleBot4
b) 프로세서: Raspberry Pi 4B 4GB(On-Board Computer)
c) 운영 체제: Ubuntu 22.04, ROS2
d) 센서:
i) 카메라: OAK-D Lite (4K 컬러/깊이) 또는 OAK-D Pro (고해상도 스테레오, IR 레이저 프로젝터)
ii) LiDAR: RPLIDAR A1M8 (360° 2D LiDAR)
iii) 기타: IMU, 휠 엔코더, 절벽 센서, 범프 센서, IR 센서
e) 배터리: 약 2-3시간

4) 충전 스테이션
a) 자율 도킹: 최소한의 수동 개입으로 AMR의 자율 도킹 및 충전 지원

3.2 소프트웨어 구성 요소

5) 공통 소프트웨어 스택
a) Ubuntu 22.04
b) ROS 2 Humble
c) Python 3
d) OpenCV, CvBridge
e) YOLOv11n / YOLOv8s

6) 모니터링 서버(PC-3)
a) Local Detection Node(Webcam, AMR 카메라 기반)
b) FLASK 기반 실시간 모니터링 UI
c) SQLite 데이터 베이스 기반 데이터 저장

7) AMR 제어 서버(PC-1, PC-2)
a) RViz2 (로봇 상태 시각화)
b) Local Detection Node (AMR 카메라 기반)
c) SLAM / Localization Node
d) Nav2

4. 데이터 흐름 설계

4.1 사전 데이터 수집

8) Webcam 에서 수집한 데이터는 YOLO11n으로 수집(빠른 디텍팅 우선) 후 대상 이미지 학습(best.pt)
9) SLAM으로 map 데이터 수집(map.pgm)

4.2 데이터 처리 및 분석

10) Webcam :학습된 데이터를 바탕으로 실시간 구역 객체 감지(일반 RGB 카메라)
11) AMR - 각 센서(RGB Camera, Depth Camera, LiDAR)로 부터 들어오는 신호를 처리하여 자율 네비게이션을 수행하고 장애물 회피 기동
12) 원격 PC - AMR로부터 카메라와 라이다, 구동부, Odometry 의 신호를 받아, 객체 감지 및 추적, 좌표 이동을 위한 연산을 진행
    a) Detection_node.py
        i) 640x640 의 사이즈로 이미지를 resize하고 oak-d 카메라 센서(stereo, rgb)를 통해 YOLO11n으로 사전 학습 시켜놓은 대상을 탐지
        ii) 탐지한 경우 /object_detected 토픽 발행
        iii) Oak-d 카메라 센서(stereo)를 통해 추적 대상과의 거리 계산(Depth 사용)
    b) Localization
        i) SLAM을 통해 만들어 놓은 map, LiDAR를 통해 발행된 /scan 과 Odometry를 통해 발행된 /odom으로 로봇 현재 위치를 Localization
        ii) 현재 위치의 좌표를 /nav_msgs_action/nav_to_pose로 토픽 발행
    c) Nav_to_pose
        i) 반대편 로봇의 협력 요청(좌표)를 /signal_amr_map으로 받아 /goal_pose로 토픽 발행
        ii) Planner : 좌표를 입력후 planner가 경로 생성
        iii) Controller : 경로를 따라 /cmd_vel 토픽을 발행하여 AMR 구동

4.3 모니터링

13) 토픽 수신 -> 화면 출력 흐름
    a) ROS2 토픽 구독 :
        i) /robot4/oakd/rgb/preview/image_raw (robot4 카메라 publish 영상)
        ii) /robot5/oakd/rgb/preview/image_raw (robot5 카메라 publish 영상)
    b) 카메라 콜백(CompressedImage, Image) :
        i) m.data를 np.frombuffer로 가져와서
        ii) cv2.imdecode를 디코딩하여 bgr로 변환
    c) Flask 엔드포인트 :
        i) get_latest(name)를 호출해서
        ii) 최신 프레임은 jpeg으로 인코딩(cv2.imencode(".jpg", frame))한 뒤
        iii) multipart/x-mixed-replace형식으로 내보냄(streaming)
        iv) HTML에서는 <img src="/video/web1"> 코드로 태그연결하여 실시간 영상 전송

5. 상세 설계

5.1 Webcam, AMR의 실시간 객체탐지 및 네비게이션

14) 객체 탐지 : 쇼핑몰 내 object(안전위협요소)를 Webcam, AMR로 각각 YOLO11n모델,

YOLOv8s를 통해 탐지 대상 인식

15) 통신 : 모니터링 pc에서 객체 감지한 데이터를 바탕으로 감지 구역의 섹션의 Waypoint, AMR의 Depth 카메라로부터 인지한 대상의 정확한 위치좌표, 감지한 대상의 클래스 종류, 상황 종료 신호 등 다양한 데이터 전송을 수행합니다
16) 이동 : 통신받은 정보를 바탕으로 섹션의 waypoint 또는 객체의 정확한 위치 좌표로 이동하는 동안 장애물을 피하며 최적의 경로로 이동합니다.

5.2 사용자 인터페이스 & 데이터베이스

17) 데이터베이스 : AMR이 객체를 감지한 시간, 객체의 종류, 객체가 최초로 발견된 섹션 위치정보가 데이터베이스에 저장됩니다
18) 모니터링 웹사이트: FLASK로 구축되었으며 Webcam-1,2와 AMR-1,2의 객체감지된 영상이 송출되며 현재 감지되고 있는 객체의 정보들이 표시됩니다.

6. 기능 요구사항

6.1 실시간 데이터 처리

19) 비디오 처리 : 각 AMR은 경고나 선택된 프레임을 중앙 서버에 전송하기 전에 비디오를 로컬에서 처리하고 분석해야 합니다
20) 위협 탐지 : 서버의 AO 모델은 여러 AMR 의 데이터를 집계하여 이벤트분리하여 패턴 확인하고 상황별 보고해야 합니다.

6.2 사용자 인터페이스 및 제어

21) UI : Flask 를 사용하여 사용자가 실시간 영상과 로그를 볼수 있도록 표시해야 합니다.

7.성능 요구사항

22) 실시간 객체 감지는 1초미만의 지연시간으로 모니터링 pc에 도달해야 합니다
23) 시스템 가동률 : AMR은 최소 4시간 동안 연속 운영이 가능해야 하며, 필요시 충전 독으로 복귀할 수 있어야 합니다.
24) 확장성: 2개의 AMR 운영에 맞춰 설계되었으나, 추가 유닛이 필요할 경우 최소한의 아키텍쳐 변경으로 지원 가능합니다.

8. 오류 처리 및 복구

25) 네트워크 손실: wifi가 끊기면 amr은제한된 기능으로 자율적으로 작동을 유지하며, 연결이 복원되면 데이터 전송을 재개 합니다.

26) 하드웨어 오류 : AMR은 정기적으로 센서 및 하드웨어 상태를 점검 합니다

27) 배터리 부족 : 배터리가 낮을경우 AMR은 자율적으로 도킹 스테이션으로 복귀합니다

9. 테스트 및 검증

28) 데이터 보호: 개인 및 비디오 데이터 처리를 위한 개인정보 또는 유사한 데이터 보호 규정 준수

10. 배포 및 유지보수 계획

29) 모든 AMR 과 중앙 서버 간의 성공적인 배포 및 원활한 통신

30) 라이브 피드 및 녹화된 클립에서 검증된 비디오 및 오디오 품질

31) 지정된 지연 내에서 보안 위협의 일관된 탐지 및 경고 생성

32) 보안 및 데이터 보호 표준 준수